

SkillPulse Kubernetes: Master Operations & Troubleshooting Guide

An end-to-end operational manual combining root cause diagnostics, command reference blueprints, background execution paradigms, and system restoration workflows

1. Root Cause Diagnostics & Architecture Validation

During local stack initialization, multi-tier microservices often encounter synchronization boundaries and system resource bottlenecks. This section analyzes the core root causes behind common local cluster initialization failures:

A. The Database Race Condition (Backend CrashLoopBackOff)

The backend microservice frequently drops into a `CrashLoopBackOff` state due to cascading loops trying to locate the target database framework (e.g., executing `Waiting for database... attempt 25/30`). This behavior triggers when the database bootstrapping pipeline outlasts the application layer fallback thresholds, or when the internal networking namespace DNS routes are misaligned.

B. The Object Resolution Boundary (Services Not Found)

Executing sweeping resource adjustments like `kubect1 delete -f .` entirely purges active object definitions from the cluster namespace. Triggering an immediate port-forward configuration afterwards returns a strict server exception: `services "frontend" not found`. No route translation pipelines can proceed until the network manifest layer is explicitly rebuilt.

C. Port Multi-Occupancy & Spontaneous Process Exits

Executing simultaneous or poorly managed port-forwards across multiple terminal sessions frequently causes immediate dropouts with layout responses such as `[1] Exit 1` or `Terminated`. This is caused by conflicting active socket allocations on your host computer (blocking local ports such as 3000 or 8080), which terminates newly created forwarding channels instantly.

2. Clean Cluster Restoration & Resource Deployment

Follow this exact chronological structure to bring the microservice infrastructure back to a clean, healthy state whenever environment inconsistencies occur:

STEP 1

Navigate to your local configuration folder within your WSL Ubuntu instance and purge all hanging socket listeners or duplicate operations:

```
cd ~/github-actions-demo/k8s
killall kubectl
pkill kubectl
```

STEP 2

Deploy all configuration manifests collectively to generate the core Namespaces, Secret records, ConfigMaps, Deployments, and Service proxies:

```
kubectl apply -f .
```

Operational Note: If individual services (such as the web user interface) fail to register due to transient file errors, target the asset configuration file directly:

```
kubectl apply -f 30-frontend.yaml
```

STEP 3

Monitor your pod statuses to verify that all functional layers have transitioned smoothly into healthy lifecycle states:

```
kubectl get pods -n skillpulse
```

VERIFIED STACK PARITY BENCHMARK:

- backend-xxxxxxx-xxxx **1/1 Running** (Ready for traffic API extraction)
- frontend-xxxxxxx-xxxx **1/1 Running** (Nginx static runtime active)
- mysql-0 **1/1 Running** (Database persistence node storage ready)

3. Localhost Connectivity & Port-Forwarding Pipelines

To access the cluster containers securely from your Windows host browser engine, select one of the following proxy deployment models:

Method A: Interactive Foreground Tunnels (Recommended for Explicit Monitoring)

This deployment path pins the execution thread directly inside an active terminal tab, making errors instantly visible. Ensure you preserve separate console tabs for each service:

Terminal Tab #1 (User Interface Routing Frame):

```
kubectl port-forward svc/frontend 3000:8080 -n skillpulse --address 0.0.0.0
```

→ *Target Web Address:* Open `http://localhost:3000` in your desktop browser.

Terminal Tab #2 (Core Logic API Framework):

```
kubectl port-forward svc/backend 8080:8080 -n skillpulse --address 0.0.0.0
```

→ *Target Backend Context:* Maps communications directly onto `http://localhost:8080`.

CRITICAL RUNTIME POLICY: Do not terminate or interrupt these dedicated terminal tabs during application testing. Closing the window drops the established network tunnel immediately.

Method B: Detached Background Daemon Operations (No Active Terminal Required)

To execute the port-forwarding proxies invisibly in the background, append output redirection operators along with the background task manager flag (`&`):

```
# Run Frontend Interface Proxy as a Background Task
kubectl port-forward svc/frontend 3000:8080 -n skillpulse --address 0.0.0.0 > /dev/null
2>&1 &

# Run Backend API Connection Bridge as a Background Task
kubectl port-forward svc/backend 8080:8080 -n skillpulse --address 0.0.0.0 > /dev/null
2>&1 &
```

System Confirmation Behavior: The operating system will output a Process ID tracker (e.g., `[1] 97548`). Once this confirms, you can safely type `exit` or close the terminal dashboard. The web interface will remain live at `http://localhost:3000`.

4. Advanced Connection Troubleshooting

If your terminal logs suggest a proxy connection is working but the page fails to load or turns blank, execute the following three verification protocols:

A. URL Verification & Protocol Validation

When the active tunnel mapping displays `Forwarding from 0.0.0.0:3000 -> 8080`, the exposed network socket is strictly accessible over port 3000. Double-check that your address layout strictly uses `http://localhost:3000`. Ensure that the browser hasn't automatically redirected the connection to `https://`, which will break the local HTTP proxy tunnel.

B. Isolate Stale Interlocking Processes

If your shell log includes hidden jobs or overlapping forwarding statements across matching ports, the socket will fail to process incoming frames. Clean out the environment by terminating active listeners completely, launch a fresh terminal screen, and execute a single standalone clean instance without background modifiers first to see if it binds successfully.

C. Service Port Configuration Auditing

Verify that your edge proxies match the internal port specifications configured on the running pod instances by running:

```
kubectl get svc frontend -n skillpulse
```

Check the `PORT(S)` configuration grid. If the internal pod is configured to listen for requests over 8080, your forwarding schema must explicitly target that port:

```
kubectl port-forward svc/frontend 3000:8080 -n skillpulse --address 0.0.0.0
```

5. Complete End-to-End Command Blueprint & Dictionary

This section documents every command used during your architecture management, debugging sessions, and container orchestration cycles, along with its specific operational purpose:

Executed Command Syntax	Operational Context & Purpose (Why It Is Used)
<pre>docker ps</pre>	Runtime Engine Verification: Checks if the Kind Kubernetes control node is actively running inside the local virtualization engine. Useful for ensuring the environment is up before sending kubectl commands.
<pre>docker start skillpulse-control-plane</pre>	Cluster Recovery Bootstrapping: Brings the stopped master Kubernetes cluster back online following a machine reboot or a virtual machine execution pause.
<pre>cd ~/github-actions-demo/k8s</pre>	Directory Scope Alignment: Shifts the working terminal context into the directory containing the cluster configuration files (.yaml) before running deployment updates.
<pre>killall kubectl pkill kubectl</pre>	Socket Lock Eradication: Forces all hidden or frozen port-forward connections to drop, releasing local ports 3000 and 8080 to prevent "address already in use" network collisions.
<pre>kubectl apply -f .</pre>	Recursive Manifest Deployment: Evaluates and builds every configuration manifest inside the current directory simultaneously to form the entire microservice stack.
<pre>kubectl apply -f 30-frontend.yaml</pre>	Isolated Resource Repair: Re-creates or updates only the frontend interface service proxy without interrupting or re-deploying any other running cluster components.
<pre>kubectl get pods -n skillpulse</pre>	Pod Lifecycle Evaluation: Displays a real-time list of all pods within the namespace to confirm they are in the <code>1/1 Running</code> state and check for any unexpected restarts.

Executed Command Syntax	Operational Context & Purpose (Why It Is Used)
<pre>kubectl get svc -n skillpulse</pre>	Service Network Auditing: Inspects active network service layers, ensuring that the necessary routing objects exist and confirming their correct internal cluster ports.
<pre>kubectl port-forward svc/frontend 3000:8080 -n skillpulse --address 0.0.0.0</pre>	UI Port Mapping Bridge: Maps your local Windows port 3000 directly to the cluster's internal port 8080, making the web frontend accessible in your browser.
<pre>kubectl port-forward svc/backend 8080:8080 -n skillpulse --address 0.0.0.0</pre>	API Stream Exposure: Connects local port 8080 to the backend service container, enabling live data communication between the frontend interface and the database.
<pre>kubectl port-forward svc/frontend 3000:8080 -n skillpulse --address 0.0.0.0 > /dev/null 2>&1 &</pre>	Asynchronous Task Daemonization: Routes all logs to a null location and detaches the proxy process from the foreground shell context. This keeps the network bridge alive even after closing the terminal window.

6. Post-Exit Resumption Workflow

When restarting your machine or starting a new development session, follow this step-by-step restoration process:

1. Confirm Docker Desktop is fully active, then run `docker start skillpulse-control-plane` to spin up the cluster nodes.
2. Clean the host networking sockets by running `killall kubectl`.
3. Launch the port-forwarding channels using either the foreground multi-tab method or the background daemon parameters detailed in Section 3.
4. Open your browser and navigate to `http://localhost:3000`. If a cached error page appears, use **Ctrl + F5** or **Ctrl + Shift + R** to trigger a hard browser refresh.